Bookshelf Project: Step 1 Setup Guide

A Goodreads-style book website built with Python, Django, and SQLite, run locally.

By the end of this step you will have a running Django project, three empty apps, a Git repository, and an admin account. Commands are given for **Windows** and **macOS/Linux** where they differ.

---

1. Install the tools

1. **Python 3.11 or newer** from python.org. On Windows, tick **"Add Python to PATH"** during installation.
2. **VS Code** from code.visualstudio.com. Add the **Python** extension.
3. **Git** from git-scm.com.

Open a terminal (VS Code's built-in one works: Terminal -> New Terminal) and verify:

```bash
python --version
git --version
```

On macOS/Linux, use `python3` if `python` isn't found, and `pip3` instead of `pip` outside the virtual environment.

---

2. Create the project folder and virtual environment

```bash
mkdir bookshelf
cd bookshelf
python -m venv venv
```

Activate it:

```bash
# Windows (PowerShell)
venv\Scripts\Activate.ps1

# Windows (Command Prompt)
venv\Scripts\activate.bat

# macOS / Linux
source venv/bin/activate
```

You should now see `(venv)` at the start of your terminal line. **Activate it every time you open a new terminal for this project.**

If PowerShell refuses with a "running scripts is disabled" error, run this once and try again:

```powershell
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

In VS Code, press `Ctrl+Shift+P` -> "Python: Select Interpreter" and choose the one inside `venv`.

Why a virtual environment? It gives this project its own isolated set of Python packages, so it doesn't clash with other projects on your computer.

---

3. Install the packages

```bash
pip install django pandas pillow
pip freeze > requirements.txt
```

- **django**: the framework
- **pandas**: for importing the book dataset in Step 3
- **pillow**: needed for profile picture uploads

`requirements.txt` records your exact versions, so you (or your marker) can recreate the setup with `pip install -r requirements.txt`.

---

4. Create the Django project

```bash
django-admin startproject config .
```

The dot at the end matters: it creates the project in the current folder instead of nesting another one. The name `config` holds your settings and main URL file.

Run the server:

```bash
python manage.py runserver
```

Open http://127.0.0.1:8000/ and you should see Django's welcome page (a rocket). Stop the server with `Ctrl+C`.

---

5. Create the three apps

```bash
python manage.py startapp accounts
python manage.py startapp books
python manage.py startapp social
```

- **accounts**: registration, login, profiles
- **books**: books, authors, genres, search
- **social**: shelves, reviews, follows, recommendations

A Django *project* is the whole website. An *app* is one self-contained section of it.

---

6. Edit the settings

Open `config/settings.py` and make these changes.

Add the apps to `INSTALLED_APPS`:

```python
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    'accounts',
    'books',
    'social',
]
```

Set up a shared templates folder. Find `TEMPLATES` and change the `DIRS` line:

```python
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Add static and media settings at the bottom of the file:

```python
```

```
STATIC_URL = 'static/'
STATICFILES_DIRS = [BASE_DIR / 'static']

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

LOGIN_URL = 'login'
LOGIN_REDIRECT_URL = 'home'
LOGOUT_REDIRECT_URL = 'home'
```

- `static` holds your CSS, JavaScript, and Chart.js file.
- `media` holds uploaded avatars.
- The three `LOGIN` lines tell Django where to send people around login and logout. The names `login` and `home` refer to URLs you'll create in later steps.

Serve uploaded files during development. Open `config/urls.py` and replace its contents with:

python

```python
from django.contrib import admin
from django.urls import path
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

---

7. Create the extra folders

bash

```bash
mkdir templates
mkdir static
mkdir media
mkdir data
```

`data` is where you'll put the downloaded book dataset in Step 3.

Your project should now look like this:

```
bookshelf/
├── venv/
├── config/
│   ├── settings.py
│   ├── urls.py
│   └── ...
├── accounts/
├── books/
├── social/
├── templates/
├── static/
├── media/
├── data/
├── manage.py
└── requirements.txt
```

---

8. Create the database and an admin account

bash

```
python manage.py migrate
python manage.py createsuperuser
```

`migrate` creates the SQLite database file (`db.sqlite3`) with Django's built-in tables, including users. `createsuperuser` asks for a username, optional email, and a password. Run the server again and visit http://127.0.0.1:8000/admin/ to log in with it. You'll use this panel constantly from Step 2 onward.

---

9. Set up Git

Create a file named `.gitignore` in the project root with this content:

```
venv/
__pycache__/
*.pyc
db.sqlite3
media/
data/*.csv
.vscode/
```

This keeps your virtual environment, database, uploads, and large dataset out of the repository. Then:

bash

```
git init
git add .
git commit -m "Initial Django project setup"
```

If Git asks who you are, set it once:

```bash
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Optional but recommended: create a free GitHub repository and push to it as a backup.

Checkpoint

All of these should be true:

- [ ] `python manage.py runserver` starts with no errors
- [ ] http://127.0.0.1:8000/ shows the Django welcome page
- [ ] You can log in at `/admin/` with your superuser
- [ ] Your folder contains `accounts`, `books`, `social`, `templates`, `static`, `media`, and `data`
- [ ] `git log` shows your first commit

What each command did

What you ran	What it does
`python -m venv venv`	Creates an isolated Python environment
`pip install django pandas pillow`	Downloads the libraries you need into that environment
`pip freeze > requirements.txt`	Saves the list of installed packages
`django-admin startproject config .`	Generates the project skeleton: settings, main URLs, `manage.py`
`python manage.py startapp books`	Creates a self-contained section of the site
`python manage.py migrate`	Turns Django's models into actual database tables
`python manage.py createsuperuser`	Creates an admin login for the admin panel
`python manage.py runserver`	Starts a local development server
Edits to `settings.py`	Tell Django which apps exist, where templates and static files live, and where to send people on login/logout

---

Self-check questions

Try answering these without looking above:

1. Why do we use a virtual environment?
2. What's the difference between a Django project and an app?
3. What does `migrate` do, and when will you need to run it again?
4. What is `settings.py` for?

Common problems

Problem	Fix
`python` or `pip` not recognised	Reinstall Python and tick "Add to PATH", or try `py` (Windows) or `python3` (macOS/Linux)
`(venv)` missing from the terminal	You forgot to activate the environment. Run the activate command again.
`ModuleNotFoundError: No module named 'django'`	The venv isn't active, or you installed Django outside it. Activate and rerun `pip install`.
`TemplateDoesNotExist` later on	Check the `DIRS` line in settings and that the `templates` folder is in the project root
Port already in use	Run `python manage.py runserver 8001`
`.gitignore` seems ignored	Make sure the file is named exactly `.gitignore` (not `.gitignore.txt`) and was created before your first `git add`

Useful resources

- Official Django tutorial, "Writing your first Django app": docs.djangoproject.com
- MDN "Django Web Framework" tutorial (builds a library website, close to this project)